

**הרצאה 09 - למידה במרחבי פעולה רציפים,
פיתוח תאורטי של Deep Deterministic Policy
Gradient (DDPG), ארכיטקטורה, עדכוני
מטרה רבים ורעש חקר.**

ד"ר יורם סגל

כל הזכויות שמורות - © ד"ר יורם סגל

June 2026

תקציר

פרק זה מהווה אבן דרך תאורטית ומעשית מרכזית בקורס למידה באמצעות חיזוקים עמוקה, ומגשר בין עולם הפעולות הבדידות והמוגבלות לעולם המורכב של הפעולות הרציפות. עד כה, נסקרו אלגוריתמים מובילים כגון רשתות קיו עמוקות ואלגוריתמי מדיניות סטוכסטיים, אשר התמודדו בהצלחה עם סביבות בעלות מספר פעולות סופי ומובחן. אולם, בעולם האמיתי, מערכות רובוטיות מתקדמות, רכבים אוטונומיים ובקרה תעשייתית דורשים שליטה במרחב פעולות רציף לחלוטין, שבו אלגוריתם מבוסס מקסימום פשוט קורס תחת הנטל החישובי. פרק זה מציג לעומק את אלגוריתם גרדיאנט המדיניות הדטרמיניסטית העמוקה, המהווה פריצת דרך תאורטית המאפשרת אופטימיזציה ישירה של מדיניות דטרמיניסטית במרחב רציף. הדיון מתחיל בסקירה מקיפה של התפתחות הלמידה מהרצאות היסוד ועד לאופטימיזציית מדיניות מקורבת, וממשיך לפירוק מתמטי ומעשי של משפט גרדיאנט המדיניות הדטרמיניסטי, כולל נוסחאות העדכון של רשתות השחקן והמבקר. הניתוח כולל את ארכיטקטורת השחקן-מבקר הייחודית של השיטה, את מנגנון העדכונים הרכים המייצב את הלמידה, ואת התפקיד הקריטי של רעש החקר באופטימיזציה דטרמיניסטית, תוך השוואה בין תהליכי רעש מורכבים לרעש גאוסיאני רגיל. כל מושג תאורטי ונוסחה מלווים בדוגמאות מספריות מלאות, הסברים ארכיטקטוניים מדויקים והצעות לתרשימי זרימה. לבסוף, הפרק מציג מטלת תכנות מעשית הדורשת פיתוח סימולטור ניווט בסיסי מאפס לשואב אבק רובוטי, תוך התבססות על מאגר הנתונים הפתוח, HouseExpo המכיל עשרות אלפי מפות של חללים פנימיים אמיתיים.

תוכן עניינים

1. מבוא: המסע לעבר בינה אוטונומית מתקדמת
2. השוואת התפתחות אלגוריתמי הלמידה
3. אתגר המרחב הרציף וקריסת השיטות המסורתיות
4. פיתוח תאורטי: מעבר לגרדיאנט מדיניות דטרמיניסטי (DPG)
5. ארכיטקטורת גרדיאנט מדיניות דטרמיניסטית עמוקה (DDPG)
6. מנגנון הייצוב באמצעות עדכוני רשתות מטרה רכים
7. פרדוקס החקר באופטימיזציה דטרמיניסטית וחשיבות הרעש
8. תהליכי האימון וההסקה הלכה למעשה: מתמטיקה ויישום
9. מילון מושגים מסכם
10. מטלת תכנות מעשית: פיתוח סימולטור רובוט מנקה מבוסס מאגר House-Expo
11. ביבליוגרפיה

1 מבוא: המסע לעבר בינה אוטונומית מתקדמת

התפתחות הבינה המלאכותית בתחום קבלת ההחלטות היא סיפור של מאבק מתמיד במורכבות העולם האמיתי. המסע הנלמד בקורס זה החל בניסיון להבין כיצד ניתן ללמד סוכן ממוחשב לקבל החלטות אופטימליות בסביבה לא ודאית, תוך התבססות על מודלים מתמטיים טהורים. בהרצאה הראשונה, הונחו היסודות המתמטיים המוצקים של התחום באמצעות ניסוח של תהליך החלטה מרקובי. תהליך זה מגדיר את העולם דרך סדרה של מצבים, פעולות, פונקציות מעבר הסתברותיות ותגמולים מספריים. משוואת בלמן הוצגה כקשר רקורסיבי אלגנטי המאפשר לחשב את הערך המדויק של כל מצב, מתוך ההנחה שתכונת מרקוב מתקיימת והעתיד תלוי אך ורק בהווה, ללא צורך בזכירת ההיסטוריה השלמה של המערכת.^[1] התפיסה הזו אפשרה פתרון של בעיות פשוטות ומוגדרות היטב באמצעות שיטות טבלאיות ותכנון דינמי, שבהן נשמר ערך מספרי לכל זוג אפשרי של מצב ופעולה. אולם, המציאות הוכיחה במהרה את מגבלותיה של גישה זו. קללת הממדיות הבהירה כי בעולם האמיתי, מספר המצבים הוא עצום, וטבלאות פשוט אינן מסוגלות לאחסן את כל המידע הנדרש מבלי לדרוש זיכרון חישובי עצום.^[1]

כדי להתגבר על מחסום זה, ההרצאות השנייה עד הרביעית תיארו קפיצת מדרגה דרמטית לעבר שילוב של למידה עמוקה. המעבר מאלגוריתמים מבוססי טבלה לקירוב פונקציות הוביל לפיתוחן של רשתות קיו עמוקות. במקום לתחזק טבלה אינסופית, רשת נוירונים עמוקה שימשה כקירוב לא-ליניארי מורכב לפונקציית הערך.^[1] אלגוריתם זה הביא עמו פריצות דרך הנדסיות ששינו את פני התחום. מנגנון מאגר השידור החוזר של חוויות שבר את המתאמים הזמניים ההרסניים בין הדגימות ואפשר למידה יציבה מתוך פיסות זיכרון אקראיות. במקביל, רשתות מטרה ייצבו את הלמידה על ידי מניעת מצב שבו המודל מנסה לפגוע במטרה שזזה ללא הרף.^[1] למרות ההצלחה הכבירה, התגלה כי שיטה זו סובלת מבעיית הערכת יתר מובנית. הפתרונות שהוצגו, סיפקו ביצועים מרשימים במרחבי פעולה בדידים.^[1]

עם זאת, התברר כי שיטות מבוססות ערך מוגבלות מעצם טבען, שכן הן דורשות חיפוש אקטיבי של פעולה הממקסמת את פונקציית הערך בכל צעד זמן. ככל שהסביבה הופכת מורכבת יותר, עלה הצורך בלמידה ישירה של המדיניות. ההרצאה החמישית והשישית סימנו שינוי פרדיגמה ומעבר לשיטות מבוססות גרדיאנט מדיניות.^[1] במקום ללמוד את שוויו של כל מצב, אומנה רשת נוירונים

להפיק ישירות התפלגות הסתברותיות של פעולות. גישה זו אפשרה התמודדות טבעית עם סביבות שבהן המדיניות האופטימלית היא סטוכסטית. כדי להפחית את השונות העצומה שאפיינה אלגוריתמים אלו, שולבה רשת נוספת ששימשה כמבקר, וכך נולדה ארכיטקטורת שחקן-מבקר. המבקר העריך את טיב המצב, והשחקן עודכן על סמך פונקציית היתרון, המייצגת את ההפרש בין התגמול בפועל לבין הציפייה המקורית של המבקר.[1]

האתגר המרכזי של שיטות גרדיאנט מדיניות נותר הרגישות הקיצונית שלהן לגודל צעד העדכון. צעד אופטימיזציה גדול מדי עלול היה להרוס לחלוטין מדיניות טובה שהושגה בעמל רב, תופעה הרסנית הידועה כקריסת מדיניות. בהרצאה השביעית נלמדו שיטות מתקדמות לייצוב הלמידה, ובראשן אופטימיזציית מדיניות מקורבת ושיטת הערכת יתרון מוכללת.[2] אלגוריתמים אלו פתרו את בעיית הרגישות על ידי יצירת פונקציית מטרה מקוצצת, המונעת מההסתברויות של המדיניות להשתנות בצורה קיצונית ביחס למדיניות הישנה, תוך יצירת אזור אמון וירטואלי המבטיח שיפור מונוטוני.[1] למרות שאלגוריתמים אלו הפכו לסטנדרט תעשייתי בשל יציבותם, הם עדיין פעלו על בסיס מדיניות סטוכסטית, אשר במקרים של בקרת מנועים רציפים דורשת דגימה מתמדת והתכנסות אטית. הרצאה זו, המהווה את לב הפרק הנוכחי, נוטשת את האקראיות המובנית ועוברת למדיניות דטרמיניסטית במרחבים רציפים, המבוססת ישירות על היתרונות ההיסטוריים של למידה מבוססת ערך.

2 השוואת התפתחות אלגוריתמי הלמידה

כדי לבסס הבנה עמוקה של הכלים שנצברו, הניתוח דורש השוואה מתודולוגית בין האלגוריתמים המובילים. הטבלה הבאה מציגה את התפתחות השיטות, סוג המרחב שהן נועדו לפתור, והיתרון המרכזי שהפך כל אחת מהן לנקודת ציון בתחום הבינה המלאכותית.

טבלה 1: השוואת אלגוריתמי למידה מובילים

שם האלגוריתם המלא	מרחב פעולות נתמך	משפחת אלגוריתמים	תכונה מרכזית ויתרון משמעותי מוביל
Q-Learning (טבלאי)	בדיד ומצומצם בלבד	מבוסס ערך טבלאי	אלגוריתם הלומד מדיניות אופטימלית ללא ידע מוקדם על מודל הסביבה. היתרון הוא פשטות היישום, אך החיסרון הוא קריסה מוחלטת במרחבי מצב גדולים עקב קללת הממדיות. [1]

(המשך בעמוד הבא)

טבלה 1: השוואת אלגוריתמי למידה מובילים (המשך טבלה)

שם האלגוריתם המלא	מרחב פעולות נתמך	משפחת אלגוריתמים	תכונה מרכזית ויתרון משמעותי מוביל
Deep Q-Network (DQN)	בדיד	מבוסס ערך עמוק	משלב רשתות נוירונים עמוקות עם מאגר שידור חוזר ורשתות מטרה. היתרון הגדול הוא פתרון קללת הממדיות של המצבים, אך הוא עדיין דורש מרחב פעולות בדיד לחלוטין. [1]
REINFORCE	בדיד ורציף	גרדיאנט מדיניות סטוכסטי	מבצע אופטימיזציה ישירה של המדיניות באמצעות רשת בודדת המפיקה התפלגות. היתרון הוא יכולת למידת מדיניות הסתברותית, אך החיסרון הוא שונות גבוהה מאוד בעדכונים. [1]

(המשך בעמוד הבא)

טבלה 1: השוואת אלגוריתמי למידה מובילים (המשך טבלה)

שם האלגוריתם המלא	מרחב פעולות נתמך	משפחת אלגוריתמים	תכונה מרכזית ויתרון משמעותי מוביל
Actor-Critic (A2C)	בדיד ורציף	שחקן-מבקר מבוסס יתרון	משלב רשת שחקן הלומדת מדיניות ורשת מבקר הלומדת פונקציית ערך. היתרון הוא הפחתה משמעותית של השונות בעזרת פונקציית היתרון, מה שמאפשר למידה יעילה וחלקה יותר. [1]
Proximal Policy Optimization (PPO)	בדיד ורציף	שחקן-מבקר סינכרוני	מייצב את שינוי המדיניות בעזרת מנגנון קיצוץ מתמטי. היתרון הוא מניעת קריסת מדיניות, יציבות יוצאת דופן ואמינות גבוהה בסביבות מגוונות. [1]

(המשך בעמוד הבא)

טבלה 1: השוואת אלגוריתמי למידה מובילים (המשך טבלה)

שם האלגוריתם המלא	מרחב פעולות נתמך	משפחת אלגוריתמים	תכונה מרכזית ויתרון משמעותי מוביל
Deep Deterministic Policy Gradient (DDPG)	רציף בלבד	שחקן-מבקר דטרמיניסטי	מפיק פעולה דטרמיניסטית ישירה במקום התפלגות הסתברותית. היתרון הוא שילוב העוצמה של למידה מבוססת ערך עם יעילות של מרחבים רציפים, תוך ניצול מקסימלי של נתוני עבר. [3]

3 אתגר המרחב הרציף וקריסת השיטות המסורתיות

הבנת הפריצה המחשבתית מאחורי אלגוריתם גרדיאנט המדיניות הדטרמיניסטית העמוקה דורשת בחינה מדוקדקת של המגבלה המובנית בשיטות מבוססות ערך מסורתיות. באלגוריתם הקלאסי המבוסס על רשתות עמוקות, רשת הנוירונים מעבדת את מצב הסביבה ומחשבת ערך מספרי עבור כל אחת מהפעולות הבדידות האפשריות. תהליך בחירת הפעולה האופטימלית בזמן אמת מתבצע על ידי סריקה של כל הפלטים וחישוב פונקציית המקסימום. הפעולה הנבחרת היא זו שממקסמת את הערכים המחושבים.

המציאות הפיזיקלית, עם זאת, אינה פועלת בקפיצות בדידות. תכנון בקר תנועה של רחפן או מערכת היגוי של שואב אבק רובוטי מציב דרישות שונות לחלוטין. במנוע חשמלי, הבקר יכול לקבל מתח עבודה רציף בטווח שבין מינוס עשרה וולט לפלוס עשרה וולט. בניגוד למשחק לוח שבו קיימות אפשרויות ברורות ומוגבלות של תנועה במרחב המשובץ, כאן המערכת יכולה להחליט על עוצמת מנוע של מספר עשרוני מדויק לחלוטין. מרחב הפעולות במקרה זה הוא אינסופי ורציף באופן מהותי. ניסיון לכפות אלגוריתם בדיד על בעיה כזו מחייב חלוקה מלאכותית של הטווח הרציף לאלפי פעולות קטנות, תהליך המכונה דיסקרטיזציה. אולם, דיסקרטיזציה של מרחב רציף רב-ממדי גורמת להתפוצצות קומבינטורית מהירה של מספר הפעולות האפשריות. אם זרוע רובוטית מכילה שבעה מפרקים נעים, ולכל מפרק מוגדרים מאה מצבי מנוע אפשריים, הרי שעבור כל צעד החלטה בודד מתקבלים מאה בחזקת שבעה צירופי פעולות. ביצוע חיפוש מקסימום על פני טריליוני אפשרויות בכל שבריר שנייה הוא פעולה שמערכות מחשוב מודרניות אינן מסוגלות לבצע בזמן אמת, והאלגוריתם נדון לקריסה חישובית מיידית.

מהצד השני של המתרחש האלגוריתמי, שיטות המבוססות על גרדיאנט מדיניות סטוכסטי מתמודדות עם מרחבים רציפים על ידי כך שהשחקן מפיק פרמטרים סטטיסטיים של התפלגות. בדרך כלל, הרשת פולטת תוחלת וסטיית תקן של התפלגות גאוסיאנית, ומתוכה מוגרלת פעולה בכל צעד. אולם, הפעולה המופקת היא מעצם טבעה הסתברותית. בבקרת תנועה פיזיקלית עדינה, לרוב לא נדרשת הגרלת פעולה מתוך ענן של אפשרויות; המערכת זקוקה לפעולה המדויקת, הדטרמיניסטית והאופטימלית ביותר האפשרית עבור אותו רגע נתון. הפתרון לכל אלו מגיע מתוך משפט מתמטי אלגנטי שמייתר לחלוטין את הצורך בהגרלות או בחיפוש מקסימום.

4 פיתוח תאורטי: מעבר לגרדיאנט מדיניות דטרמיניסטי (DPG)

בשנת אלפיים וארבע עשרה, חוקר הבינה המלאכותית דיוויד סילבר יחד עם עמיתיו הציגו לעולם את משפט גרדיאנט המדיניות הדטרמיניסטי. [4] עד לאותו מועד מכונן, הקהילה המדעית החזיקה בתפיסה כי אופטימיזציה של מדיניות חייבת להתבצע דרך חקר סטוכסטי. ההנחה הייתה שאם המדיניות היא דטרמיניסטית, כלומר פולטת פלט קבוע וזהה תמיד עבור מצב נתון, הרי שהגרדיאנט של פונקציית המטרה ביחס להסתברויות הפעולה פשוט אינו מוגדר מבחינה מתמטית רציפה.

ההוכחה האלגנטית של סילבר שינתה את חוקי המשחק. היא קבעה כי אם מגדירים את המדיניות כפונקציה דטרמיניסטית הממפה מצב ישירות לפעולה רציפה, ניתן לעדכן את המשקולות שלה בצורה ישירה. העדכון מתבצע על ידי שימוש בכלל השרשרת מתוך החשבון הדיפרנציאלי.

הנוסחה המכוננת של משפט זה ניתנת לתיאור מדויק באופן הבא: נגדיר את פונקציית המטרה שלנו כתוחלת התגמול הכולל, ונסמנה באות הלועזית J . המדיניות הדטרמיניסטית שלנו תסומן כ- $\mu_\theta(s)$, כאשר θ מייצג את משקולות רשת השחקן. ערך הפעולה יעורך על ידי המבקר שנסמנו $Q^\mu(s, a)$. המשפט קובע כי הגרדיאנט של המטרה מתקבל מתוך משוואה (1): [4]

$$(1) \quad \nabla_\theta J(\mu_\theta) = \mathbb{E}_{s \sim \rho^\mu} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q^\mu(s, a) \Big|_{a=\mu_\theta(s)} \right]$$

במילים פשוטות, כדי לדעת לאיזה כיוון לעדכן את הרשת, אנו מכפילים שני שיפועים: האיבר הימני, $\nabla_a Q^\mu(s, a)$, הוא הנגזרת של המבקר ביחס לפעולה. הוא שואל את המבקר: "אם הייתי משנה את הפעולה קצת, איך זה היה משפיע על ציון הערך הסופי?". האיבר השמאלי, $\nabla_\theta \mu_\theta(s)$, הוא הנגזרת של רשת השחקן ביחס למשקולותיה, והוא עונה על השאלה: "איך עלי לשנות את המשקולות כדי להזיז את הפעולה בדיוק בכיוון שהמבקר ביקש?". [4]

דוגמה מספרית לחשבון הגרדיאנט:

יש לדמיין רובוט הנע על ציר ישר. המצב מתואר על ידי המרחק שלו מהקיר, שהוא 5 מטרים. רשת השחקן, במצבה הנוכחי, פולטת פקודת מהירות של 2 מטרים לשנייה. אנו שואלים את המבקר: "מה יקרה אם נגדיל את המהירות במעט?". המבקר מחשב את הנגזרת ביחס לפעולה ומחזיר ערך של 0.5. משמעות הדבר היא שעלייה קטנה במהירות תשפר את הציון הכולל. כעת,

אנו מסתכלים על נגזרת רשת השחקן ביחס למשקולת פנימית מסוימת, ונניח שהיא 0.8. המכפלה של השניים היא $0.4 = 0.8 \times 0.5$. הגרדיאנט הכולל עבור אותה משקולת הוא 0.4. לכן, נעדכן את המשקולת כלפי מעלה כדי להגביר את המהירות במצב זה בפעם הבאה. התהליך כולו דטרמיניסטי, מחושב וישיר.

5 ארכיטקטורת גרדיאנט מדיניות דטרמיניסטית עמוקה (DDPG)

שנה לאחר פרסום המשפט התאורטי, קבוצת חוקרים שילבה את ההוכחות המתמטיות של גרדיאנט המדיניות הדטרמיניסטי עם היכולות של רשתות נוירונים עמוקות, ויצרה את האלגוריתם המוכר כיום כ-Deep Deterministic Policy Gradient [3]. החוקרים הבינו שכדי לגרום לאלגוריתם לפעול בהצלחה בסביבות מורכבות, עליהם לאמץ את טכניקות הייצוב שפותחו עבור רשתות קיו עמוקות, ולהטמיע אותן בתוך מבנה השחקן-מבקר.

הארכיטקטורה המלאה מבוססת על שתי ישויות מרכזיות, כאשר כל אחת מיוצגת על ידי רשת נוירונים עמוקה עם פרמטרים שונים. נסמן את פרמטרי רשת השחקן ב- θ^μ , ואת פרמטרי רשת המבקר ב- θ^Q [3].

תחילה נבחן את רשת השחקן $\mu(s | \theta^\mu)$. הכניסה לרשת זו היא וקטור המצב של הסביבה s . המוצא של רשת השחקן הוא הפעולה הרציפה a . כדי לוודא שהפעולה אינה חורגת מהמגבלות הפיזיקליות של המערכת, השכבה האחרונה של הרשת מעבירה את התוצאה דרך פונקציית אקטיבציה היפרבולית (Tanh) שדוחסת את הפלט לטווח רציף שבין 1 ל-1.

דוגמה מספרית לפעולת השחקן:

נניח שמצב הסביבה מתואר על ידי וקטור של שני ערכים: המרחק למכשול הוא 4.0 והמהירות היא 1.5. אנו מזינים את הווקטור $[1.5, 4.0]$ לתוך רשת השחקן. לאחר מעבר בכל השכבות והכפלה במשקולות, ערך המוצא לפני האקטיבציה הוא 0.9. פונקציית ה-Tanh מופעלת על 0.9 ומחזירה בערך 0.71. לכן, הפעולה הדטרמיניסטית שתבוצע היא 0.71, אשר יכולה לייצג 71 אחוזים מעוצמת המנוע המרבית קדימה.

כעת נבחן את רשת המבקר $Q(s, a | \theta^Q)$. רשת זו אינה מסתפקת רק בנתוני הסביבה, אלא מקבלת כניסה משולבת (Concatenation) של וקטור המצב

s ווקטור הפעולה הרציפה a שהופקה על ידי השחקן. המוצא של המבקר הוא סקלאר בודד המייצג את ערך המצב-פעולה (Q-value).

דוגמה מספרית לפעולת המבקר:

המבקר מקבל את וקטור המצב המקורי $[1.5, 4.0]$ ואת הפעולה שהשחקן הציע, שהיא 0.71 . הוא משרשר אותם לווקטור אחד: $[0.71, 1.5, 4.0]$. וקטור זה מוזן לתוך רשת המבקר. המבקר מעבד את הנתונים ופולט ערך יחיד, למשל 15.5 . משמעות הדבר היא שהמערכת צופה שהביצוע של עוצמת מנוע 0.71 במצב הנתון יוביל לפרס מצטבר עתידי של 15.5 נקודות.

תיאור מדויק של סכימת הבלוקים למערכת זו יראה כך באנגלית עבור שרטוט תרשימים: הסכימה המרכזית צריכה להיות מוכתרת בכותרת "DDPG Architecture". בצידה השמאלי ימוקם בלוק הנקרא " s State". מתוך בלוק זה ייצא חץ ישירות לבלוק עליון בשם " $\mu(s)$ Actor Network", אשר ממנו ייצא חץ פלט בשם " a Action". במקביל לכך, חץ מתוך בלוק " s State" וחץ מתוך ה-" a Action" ייכללו יחד וייכנסו לבלוק תחתון בשם " $Q(s, a)$ Critic Network". מתוך המבקר ייצא חץ פלט סופי בשם "Q-value".

6 מנגנון הייצוב באמצעות עדכוני רשתות מטרה רכים

מכיוון שרשת המבקר מעדכנת את משקולותיה על בסיס הערכות עתידיות שמופקות על ידי עצמה, נוצרת לולאת משוב בלתי יציבה. הרשת רודפת אחרי תחזיותיה שלה, תהליך המוביל לתנודות הרסניות. באלגוריתמים קודמים בעיה זו נפתרה על ידי הקפאה מוחלטת של רשת המטרה, והעתקת המשקולות אחת לאלפי צעדים. [1]

אולם, ב-DDPG, העתקה אגרסיבית גורמת לזעזועים. הפתרון הוא שימוש בעדכוני מטרה רכים (Soft Target Updates). המערכת מחזיקה בזיכרון רשתות שחקן ומבקר מקוונות (θ^μ ו- θ^Q), ובמקביל רשתות מטרה תואמות (θ^μ ו- θ^Q). במקום להעתיק את המשקולות בבת אחת, מעדכנים את רשתות המטרה בכל צעד אימון בודד באטיות, לפי פרמטר τ קטן (לרוב 0.001), כמוצג במשוואות (2) ו-(3):

$$(2) \quad \theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$(3) \quad \theta^{\mu'} \leftarrow \tau \theta^{\mu} + (1 - \tau) \theta^{\mu'}$$

דוגמה מספרית לעדכון רך:

נניח שלשחקן המקוון יש משקולת בודדת שערכה המעודכן הוא 10. למשקולת המקבילה ברשת שחקן המטרה יש ערך של 4. קבוע העדכון τ מוגדר כ-0.01.

נציב בנוסחה:

הערך החדש של משקולת המטרה יהיה:

$$0.01 \times 10 + 0.99 \times 4 = 0.1 + 3.96 = 4.06$$

רשת המטרה נעה בעדינות אסימפטוטית מארבע לערך של 4.06, מה שמבטיח שיעדי הלמידה ישתנו ביציבות.

7 פרדוקס החקר באופטימיזציה דטרמיניסטית וחשיבות הרעש

כאשר המדיניות היא דטרמיניסטית, רשת השחקן תמיד תפיק בדיוק את אותו ערך מספרי עבור מצב נתון. הסוכן לעולם לא ינסה אפשרויות חדשות ויתקע באסטרטגיה הראשונה שמצא. הפתרון במרחב רציף טמון בהוספה אקטיבית ומלאכותית של רעש לחקר (Exploration Noise) לפלט השחקן במהלך איסוף הנתונים בלבד, כמתואר במשוואה (4):

$$(4) \quad a_t = \mu(s_t | \theta^{\mu}) + \mathcal{N}$$

במאמר המקורי, החוקרים השתמשו בתהליך סטוכסטי מורכב הנקרא Ornstein-Uhlenbeck Process, (OU) המייצר רעש בעל מתאם זמני, תכונה שימושית למערכות פיזיקליות עם התמדה. אולם, סקירות מחקריות מודרניות הוכיחו כי למורכבות החישובית של תהליכים אלו לרוב אין הצדקה גורפת. בפועל, רעש גאוסיאני (Gaussian Noise) פשוט, הנדגם מהתפלגות נורמלית חסרת תלות, מספק ביצועים מעולים ופשוט בהרבה ליישום. [3]

דוגמה מספרית לחקר גאוסיאני:

נניח שהשחקן הפיק פקודת היגוי אידיאלית של 0.4. המערכת מגרילה רעש מהתפלגות נורמלית עם תוחלת 0 וסטטיית תקן של 0.15. תוצאת ההגרלה היא מינוס 0.15. הפעולה שתבצע בפועל בסביבה היא החיבור שלהם:

$$a_t = 0.4 + (-0.15) = 0.25$$

הרובוט ביצע תנועה שונה מהמתוכנן, מה שמאפשר לו לחקור נתיבים חדשים שיישמרו במאגר הזיכרון ללמידה עתידית.

8 תהליכי האימון וההסקה הלכה למעשה: מתמטיקה ויישום

לאחר ביסוס אבני הבניין, נתאר את מחזור הלמידה השלם. המערכת מבצעת פעולות בסביבה ושומרת רביעיות נתונים של מצב, פעולה, תגמול ומצב חדש (s_i, a_i, r_i, s_{i+1}) בתוך מאגר הזיכרון (Replay Buffer). כאשר המאגר מלא מספיק, שולפים ממנו אצווה (Batch) אקראית של נתונים, ומתחילים באופטימיזציה.

חישוב המטרה (Target) עבור המבקר:

המערכת צריכה להגדיר מהו היעד הנכון שהמבקר צריך לשאוף אליו. לשם כך, אנו מעבירים את המצב הבא s_{i+1} אל רשת שחקן המטרה הקפואה, כדי להבין איזו פעולה נבצע בעתיד. את המצב העתידי והפעולה העתידית אנו מעבירים למבקר המטרה הקפואה, ומקבלים הערכה עתידית. הנוסחה המדויקת ליעד y_i נתונה במשוואה (5):

$$(5) \quad y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1} | \theta^{\mu'}) | \theta^{Q'})$$

עדכון המבקר (Critic Update):

לאחר שחישבנו את היעד y_i , אנו מבקשים מרשת המבקר המקוונת שלנו לתת את ההערכה הנוכחית שלה לאותו מצב ופעולה מהעבר, $Q(s_i, a_i | \theta^Q)$. פונקציית ההפסד (Loss) היא פשוט השגיאה הריבועית הממוצעת ביניהם, כמוגדר במשוואה (6):

$$(6) \quad L(\theta^Q) = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i | \theta^Q))^2$$

דוגמה מספרית מלאה לעדכון המבקר (עבור דגימה אחת, $N = 1$):

נניח שדגמנו חוויה בה התגמול המיידי r_i היה 10 נקודות. פקטור ההיוון γ הוא 0.9. אנו מזינים את המצב העתידי s_{i+1} לשחקן המטרה, והוא בוחר בפעולה 0.5. המצב העתידי והפעולה העתידית מוזנים למבקר המטרה, שמחשב להם שווי עתידי של 20. נחשב את היעד:

$$y_i = 10 + 0.9 \times 20 = 10 + 18 = 28$$

היעד האמיתי שלנו הוא 28. כעת אנו מעבירים את המצב והפעולה מהחוויה המקורית למבקר הלומד, והוא מנבא שווי של 25. נחשב את ההפסד (Loss):

$$L = (28 - 25)^2 = 3^2 = 9$$

אלגוריתם האופטימיזציה ישנה את משקולות המבקר כך שהתחזית שלו תעלה מ-25 לכיוון ה-28.

עדכון השחקן (Actor Update): כדי לעדכן את השחקן, אנו רוצים למקסם את ערך ה-Q שהמבקר מחזיר. אנו מעבירים את המצב המקורי דרך השחקן המקוון, לוקחים את הפעולה החדשה שלו, ומזינים אותה למבקר. הגרדיאנט המדויק מחושב לפי משוואה (7): [4]

$$(7) \quad \nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a | \theta^Q) \Big|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) \Big|_{s=s_i}$$

דוגמה מספרית לעדכון השחקן:

השחקן הלומד רואה את המצב s_i ומציע פעולה בעוצמה 0.4. אנו שואלים את המבקר המקוון על הצירוף הזה, והוא קובע שהגרדיאנט ביחס לפעולה

הוא 2.0. כלומר, עלייה של יחידה אחת בפעולה תשפר את הציון ב-2 נקודות. נגזרת המשקולת של השחקן ביחס לפעולה היא 1.5. לכן, על ידי הכפלתם $(3.0 = 1.5 \times 2.0)$, הרשת תעדכן את משקולת השחקן בערך חיובי, כך שבפעם הבאה הוא יציע עוצמה מעט גבוהה יותר מ-0.4.

בשלב ההסקה הסופי (Inference) בעולם האמיתי, אנו זורקים את רשתות המטרה ואת המבקר. נשאר רק עם השחקן המקוון והדטרמיניסטי. הרובוט מזין את המצב לרשת מקבל פעולה נקייה ללא רעש, ומבצע אותה ביעילות שיא.

9 מילון מושגים מסכם

טבלה 2: מילון מושגים מסכם

מושג מקצועי	הגדרה והסבר מפורט
MDP (תהליך החלטה מרקובי)	תשתית מתמטית למידול בעיות של קבלת החלטות, המורכבת ממצבים, פעולות ותגמולים. [1]
DPG (גרדיאנט מדיניות דטרמיניסטי)	משפט תאורטי הקובע שניתן לחשב גרדיאנט מדיניות ישירות על ידי כפל הנגזרת של המבקר בנגזרת של השחקן, ללא צורך בהתפלגויות. [4]
DDPG (גרדיאנט מדיניות דטרמיניסטית עמוקה)	אלגוריתם מחוץ-למדיניות המיישם את DPG בעזרת רשתות נוירונים עמוקות, מיועד לבעיות במרחבים רציפים. [3]
Actor (רשת השחקן)	רשת הפולטת פקודת פעולה דטרמיניסטית (כגון Tanh), המסומנת כ- $\mu(s \theta^\mu)$.

(המשך בעמוד הבא)

טבלה 2: מילון מושגים מסכם (המשך טבלה)

מושג מקצועי	הגדרה והסבר מפורט
Critic (רשת המבקר)	רשת הקולטת מצב סביבה ופעולה, ומחשבת את הציון (Q-value), מסומנת כ- $Q(s, a \theta^Q)$.
Soft Target Updates	טכניקת ייצוב המעדכנת את רשתות המטרה באחוז זעיר (τ) לעבר הרשתות הלומדות בכל צעד.
Exploration Noise	תוספת סטוכסטית (גאוסיאנית או OU) המוזרקת לפלט השחקן בזמן אימון כדי לאלץ חקר סביבה.
Replay Buffer	מערך זיכרון לאגירת חוויות העבר, המאפשר דגימת אצווה לשבירת מתאמים זמניים. [1]

10 מטלת תכנות מעשית: פיתוח סימולטור רובוט מנקה ויישום DDPG

לשם יישום מעשי של התיאוריה שנלמדה בשיעור, הסטודנטים נדרשים לבצע מטלת תכנות מלאה המבוססת על פלטפורמות קידוד ברשת יחד עם עוזרי בינה מלאכותית. מטרת המטלה היא התנסות חווייתית בפיתוח סוכן המסוגל ללמוד ולנווט במרחב רציף, בסביבה המדמה ניווט אוטונומי של שואב אבק רובוטי. הדגש המרכזי במטלה הוא היישום ההלכה למעשה של אלגוריתם ה-DDPG שנלמד בהרצאה, והבנת היתרונות שלו על פני אלגוריתמים קודמים.

דרישת חובה: איסור על שימוש בפלטפורמות סימולציה מוכנות.

בניגוד לתרגילים קודמים, כאן **אסור לחלוטין** להשתמש בסביבות מוכנות כמו Gymnasium או Gazebo. על הסטודנטים לייצר מאפס סימולטור דו-ממדי בסיסי ופשוט של קינמטיקה, אשר קורא מפות חדרים אמיתיות.

בסיס הנתונים: HouseExpo

לבניית הסימולטור, הסטודנטים יורידו וישתמשו במאגר הנתונים "House-Expo" [5]. מאגר אקדמי פתוח זה מכיל למעלה מ-35,000 תוכניות רצפה דו-ממדיות בפורמט JSON, של בתים אמיתיים ומורכבים, הכוללים חדרים, קירות ופתחים.

קישור מדויק למאגר ב-GitHub: ([opxEesuoH/iLnagaeT/moc.buhtig//sptth](https://github.com/pxEesuoH/iLnagaeT/moc.buhtig//sptth))

הגדרת הבעיה והקשר הישיר לשיעור (DDPG):

במשימה זו, בניגוד למשחקי לוח, מרחב הפעולות הוא רציף לחלוטין. פעולה (Action) מוגדרת כווקטור רציף דו-ממדי בין 1-1 המייצג את עוצמת ההנעה הישירה של המנועים המרכזיים קדימה או אחורה, וכן את רדיוס ועוצמת פקודת ההיגוי. כפי שנלמד בהרצאה, שימוש באלגוריתמים מבוססי פעולות בדידות (כמו DQN) ידרוש חלוקה מלאכותית (דיסקרטיזציה) של המרחב הרציף, מה שיוביל מהר מאוד לפיצוץ קומבינטורלי. לכן, יישום DDPG כאן הוא הכרחי משום שהוא פותר בעיות שליטה רציפות באופן טבעי.

- **מצב (State):** וקטור רציף המורכב מסדרת קריאות חיישני מרחק המחושבים וירטואלית מול קירות מפת ה-JSON, וכן נתוני המהירות.

- **תגמול (Reward):** תגמול חיובי על ניקוי אזורים חדשים במפה, וקנס שלילי על התנגשויות.

דרישות המימוש, התיעוד וההגשה בקוד:

על הסטודנטים להראות בבירור במסמך ההגשה ובקוד עצמו (באמצעות הערות קוד מפורטות) היכן באים לידי ביטוי הרכיבים הספציפיים של DDPG. חובה לכלול במסמך הסיכום סעיף המפרט את הדברים הבאים:

1. **ארכיטקטורת שחקן-מבקר:** ציינו היכן בקוד מוגדרת רשת השחקן (הפולטת פעולה דטרמיניסטית התחומה על ידי פונקציית Tanh) והיכן רשת המבקר (הקולטת גם את המצב וגם את הפעולה).
2. **עדכוני רכיבים (Soft Target Updates):** הציגו את שורות הקוד המדויקות המבצעות את מיצוע הפוליאק לעדכון רשתות המטרה.
3. **פרמטרים והיפר-פרמטרים:** פרטו במסמך את כל הפרמטרים שבחרתם לאימון: מהו קצב הלמידה של השחקן ושל המבקר? מהו הערך שנבחר ל- τ (למשל 0.005)? מהו גורם ההיוון γ ? הסבירו את בחירתכם.
4. **רעש חקר (Exploration Noise):** הסבירו היכן מתווסף הרעש הגאוסיאני לקוד המבצע את איסוף הנתונים מהסימולטור, ומהי השונות הראשונית

שנבחרה.

גרפים, הצגת נתונים וויזואליזציה:

ללא הצגת תוצאות ברורות, האימון אינו נחשב כמושלם. הדרישות להגשה כוללות:

- **גרפים של התכנסות ואימון:** חובה להציג שני גרפים מרכזיים:

א. גרף "עקומת הלמידה" (Learning Curve) המציג את התגמול המצטבר (Re-ward) לאורך האפיזודות.

ב. גרף של שגיאת המבקר (Critic Loss) לאורך זמן ההתכנסות.

- **תצוגה ויזואלית של פעולת הרובוט:** בדומה ל"משימת הרחפנים" שקיבלתם בעבר, חובה למצוא דרך ויזואלית להציג את פעולת הרובוט המנקה הלכה למעשה. ניתן לייצר סרטון אנימציה קצר או פשוט לשרטט קווי מסלול (Trajectory) בצבע על גבי מפת ה-JSON הדו-ממדית, המוכיחים כיצד השואב מתחמק מקירות ומכסה את שטחי החדרים באופן חלק ורציף.

שאלות ניתוח והבנה:

בדוח המסכם של הפרויקט, עליכם לענות במדויק על השאלות הבאות, המראות שהבנתם את הקשר בין האלגוריתם למשימה:

1. מדוע בחרנו דווקא באלגוריתם DDPG למשימת ניווט השואב, ולא באלגוריתם DQN טבלאי עמוק או ב-PPO? (רמז: התייחסו לאופי הדטרמיניסטי של המנועים הפיזיים ולניצול נתונים ממאגר).
2. מה היה קורה אם הייתם מסירים לחלוטין את הוספת הרעש הגאוסיאני (Exploration Noise) מהפלט של רשת השחקן בשלבי האימון הראשוניים? הסבירו כיצד זה היה משפיע על מפת אזורי הניקוי של השואב במבוך.
3. כיצד מנגנון רשתות המטרה והעדכונים הרכים (Soft Target Updates) הגן על רשת המבקר של השואב מפני קריסה לאורך האימון?

מקורות בעברית

- 1 י. סגל, הרצאה 10: ממשוואת בלמן ועד למידה עמוקה, הערות הרצאה, קורס למידה באמצעות חיזוקים עמוקה, אוניברסיטת בר-אילן, 2026.
- 2 י. סגל, הרצאה 70: אופטימיזציה מדיניות מקורבת (PPO) ושיטת GAE, הערות הרצאה, קורס למידה באמצעות חיזוקים עמוקה, אוניברסיטת בר-אילן, 2026.

English References

- 3 T. P. Lillicrap et al., “Continuous control with deep reinforcement learning,” *arXiv preprint arXiv:1509.02971*, 2015. [Online]. Available: <https://arxiv.org/abs/1509.02971>
- 4 D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, “Deterministic policy gradient algorithms,” in *Proceedings of the 31st International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, 32, PMLR, 2014, 387–395. [Online]. Available: <https://proceedings.mlr.press/v32/silver14.pdf>
- 5 T. Li, D. Ho, C. Li, D. Zhu, C. Wang, and M. Q.-H. Meng, “HouseExpo: A large-scale 2D indoor layout dataset for learning-based algorithms on mobile robots,” *arXiv preprint arXiv:1903.09845*, 2019. [Online]. Available: <https://github.com/TeaganLi/HouseExpo>